0–1 KNAPSACK USING DYNAMIC PROGRAMMING

Design and Analysis of Algorithms (DAA) — C Implementation

1. Aim

To implement and execute the 0–1 Knapsack problem using dynamic programming in C and find the maximum total value within a given capacity.

2. Theory

Each item can either be selected once or not selected. An item cannot be divided or reused. Dynamic programming solves smaller subproblems and stores their answers in a table.

3. Algorithm

1. Create a table dp[n+1][W+1].
2. Set row 0 and column 0 to 0.
3. For each item i and capacity w, skip the item if its weight is too large.
4. Otherwise choose the maximum of taking it or leaving it.
5. The answer is dp[n][W]. Trace the table to identify selected items.

4. Example Input

Item	Weight	Value
1	1	1
2	2	6
3	3	10
4	5	16

Knapsack capacity W = 7

5. DP Table / Graph

0–1 Knapsack Dynamic Programming Table (Capacity = 7)

	0	1	2	3	4	5	6	7
0 items	0	0	0	0	0	0	0	0
Item 1	0	1	1	1	1	1	1	1
Item 2	0	1	6	7	7	7	7	7
Item 3	0	1	6	10	11	16	17	17
Item 4	0	1	6	10	11	16	17	22

Each cell dp[i][w] shows the best value achievable using the first i items with capacity w. The final answer is the bottom-right cell.

6. C Program (Zoomed Code)

The C code is shown in a larger font with extra line spacing for easy reading and printing.

```
#include <stdio.h>

#define N 4
#define W 7

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main(void) {
    int weight[N] = {1, 2, 3, 5};
    int value[N]  = {1, 6, 10, 16};

    /* dp[i][w] = best value using first i items
       with knapsack capacity w */
    int dp[N + 1][W + 1];

    for (int i = 0; i <= N; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            } else if (weight[i - 1] <= w) {
                dp[i][w] = max(
                    value[i - 1] +
                    dp[i - 1][w - weight[i - 1]],
                    dp[i - 1][w]
```

6. C Program (continued)

```
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    printf("Maximum value = %d\\n", dp[N][W]);

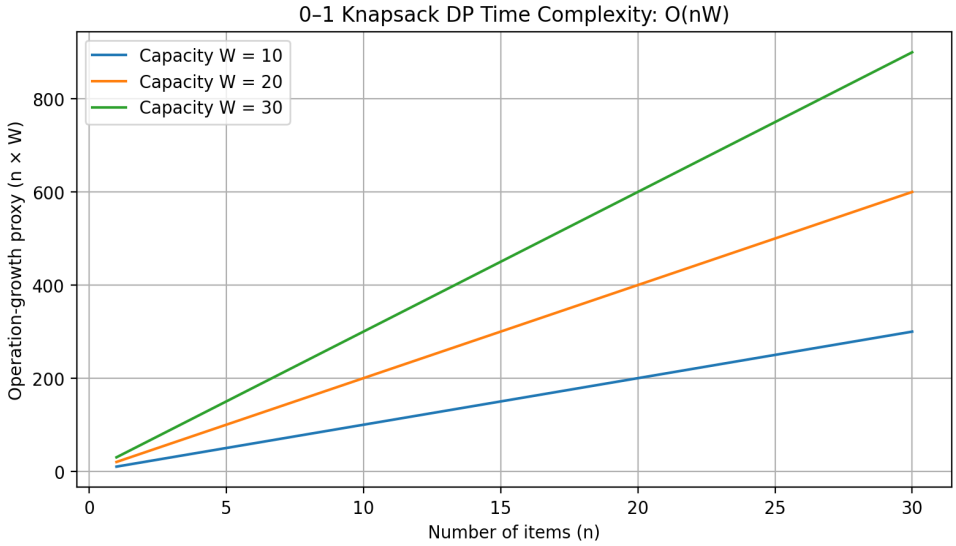
    /* Print selected items by tracing the DP table */
    int w = W;
    printf("Selected items (1-based): ");
    for (int i = N; i > 0 && w > 0; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            printf("%d ", i);
            w -= weight[i - 1];
        }
    }
    printf("\\n");
    return 0;
}
```

7. Sample Output

```
Maximum value = 22
Selected items (1-based): 4 2
```

8. Time and Space Complexity

Measure	Complexity
Time complexity	O(nW)
Space complexity (2D DP table)	O(nW)



The complexity depends on both the number of items n and the capacity W; this is called pseudo-polynomial time.

9. Result and Conclusion

The program finds a maximum value of 22 for capacity 7 by selecting items 2 and 4 (total weight 2 + 5 = 7, total value 6 + 16 = 22). The 0–1 Knapsack problem is solved using dynamic programming with O(nW) time and space.